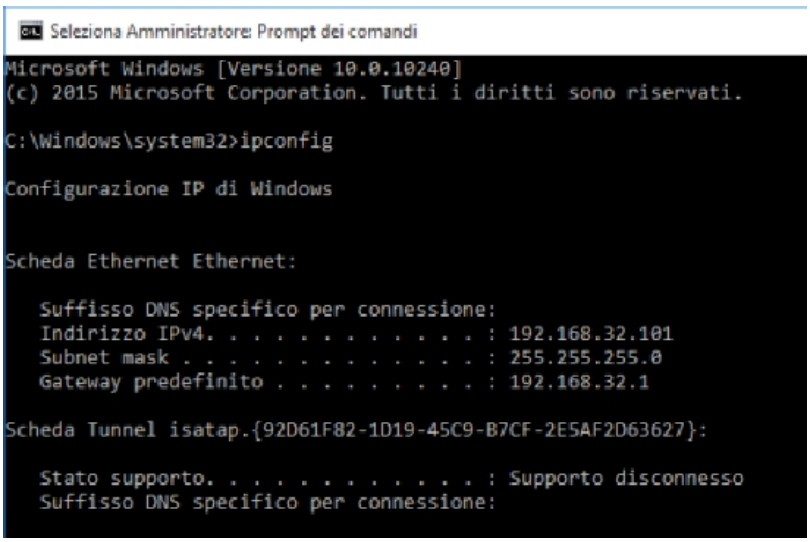
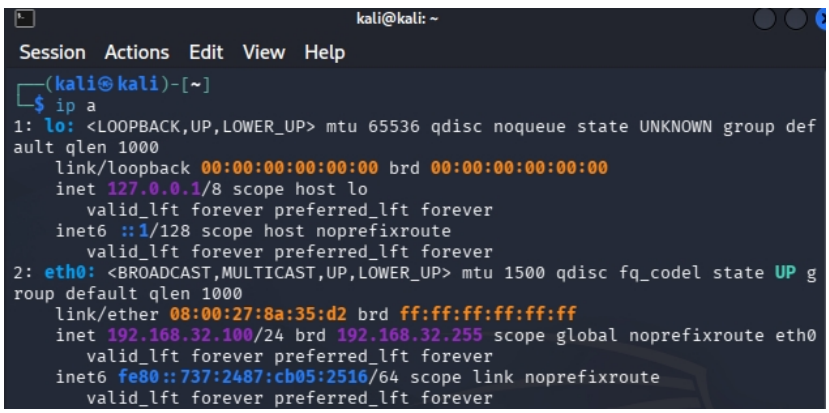
**PROGETTO FINALE - W4D4**

Luigi Valerio Pancotti

Epicode

Screening lavoro (a seguire la spiegazione del lavoro svolto):



```
┌──(kali㉿kali)-[~]
└─$ sudo inetsim
[sudo] password for kali:
INetSim 1.3.2 (2020-05-19) by Matthias Eckert & Thomas Hungenberg
Using log directory:      /var/log/inetsim/
Using data directory:     /var/lib/inetsim/
Using report directory:   /var/log/inetsim/report/
Using configuration file: /etc/inetsim/inetsim.conf
Parsing configuration file.
Configuration file parsed successfully.
=== INetSim main process started (PID 37097) ===
Session ID:     37097
Listening on:   127.0.0.1
Real Date/Time: 2026-04-30 13:22:40
Fake Date/Time: 2026-04-30 13:22:40 (Delta: 0 seconds)
 Forking services...
  * https_443_tcp - started (PID 37099)
 done.
Simulation running.
```

```
▾ Internet Protocol Version 4, Src: 192.168.32.101, Dst: 192.168.32.100
    0100 .... = Version: 4
    .... 0101 = Header Length: 20 bytes (5)
  ▸ Differentiated Services Field: 0x00 (DSCP: CS0, ECN: Not-ECT)
    Total Length: 1877
    Identification: 0x139e (5022)
  ▸ 010. .... = Flags: 0x2, Don't fragment
    ...0 0000 0000 0000 = Fragment Offset: 0
    Time to Live: 128
    Protocol: TCP (6)
    Header Checksum: 0x1deb [validation disabled]
    [Header checksum status: Unverified]
    Source Address: 192.168.32.101
    Destination Address: 192.168.32.100
    [Stream index: 0]
```

```
▾ Ethernet II, Src: PCSSystemtec_15:0f:9c (08:00:27:15:0f:9c), Dst: PCSSystemtec_8a:35:d2 (08:00:27:8
  ▾ Destination: PCSSystemtec_8a:35:d2 (08:00:27:8a:35:d2)
      .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
      .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
  ▾ Source: PCSSystemtec_15:0f:9c (08:00:27:15:0f:9c)
      .... ..0. .... .... .... .... = LG bit: Globally unique address (factory default)
      .... ...0 .... .... .... .... = IG bit: Individual address (unicast)
    Type: IPv4 (0x0800)
    [Stream index: 0]
```

```
Handshake Protocol: Client Hello
  Handshake Type: Client Hello (1)
  Length: 1828
▸ Version: TLS 1.2 (0x0303)
  Random: 7d52192b3eb37c32b7799a4e2d4bbf12a6c706c4f17a5e6a77477c681de27919
  Session ID Length: 32
  Session ID: cb5a63f6103f7a310ec5eeb8ce840d9df3f938b10403f398e7d14676e36029fa
  Cipher Suites Length: 32
```

```
▾ Hypertext Transfer Protocol
  ▸ GET / HTTP/1.1\r\n
    Host: epicode.internal\r\n
    Connection: keep-alive\r\n
    Cache-Control: max-age=0\r\n
    Upgrade-Insecure-Requests: 1\r\n
    User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/126.0.0.0
    Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,appl
    Accept-Encoding: gzip, deflate\r\n
    Accept-Language: it-IT,it;q=0.9,en-US;q=0.8,en;q=0.7\r\n
    If-None-Match: "29cf-64d6f06ad6fbf-gzip"\r\n
    If-Modified-Since: Fri, 20 Mar 2026 06:45:09 GMT\r\n
    \r\n
    [Response in frame: 18]
    [Full request URI: http://epicode.internal/]
```

**Report Tecnico: Analisi del Traffico di Rete HTTP vs HTTPS**

## 1 Introduzione

Il presente documento descrive le attività di simulazione e analisi del traffico di rete tra un client Windows e un server Kali Linux. L'obiettivo è evidenziare le differenze di sicurezza tra i protocolli **HTTP** e **HTTPS** attraverso l'intercettazione dei pacchetti e l'analisi dei payload.

L'architettura client-server è stata configurata in un ambiente virtuale isolato con i seguenti parametri:

- **Server (Kali Linux):**
- **Indirizzo IP:** 192.168.32.100 (Verificato tramite comando ip a).
- **Indirizzo MAC:** 08:00:27:8a:35:d2.
- **Servizi:** Simulazione di rete tramite **INetSim**, configurato per rispondere all'hostname epicode.internal.
- **Client (Windows):**
- **Indirizzo IP:** 192.168.32.101 (Verificato tramite ipconfig).
- **Indirizzo MAC:** 08:00:27:15:0f:9c.

## 2. Configurazione dell'Ambiente di Laboratorio

L'architettura client-server è stata configurata in un ambiente virtuale isolato con i seguenti parametri:

- **Server (Kali Linux):**
- **Indirizzo IP:** 192.168.32.100 (Verificato tramite comando ip a).
- **Indirizzo MAC:** 08:00:27:8a:35:d2.
- **Servizi:** Simulazione di rete tramite **INetSim**, configurato per rispondere all'hostname epicode.internal.
- **Client (Windows):**
- **Indirizzo IP:** 192.168.32.101 (Verificato tramite ipconfig).
- **Indirizzo MAC:** 08:00:27:15:0f:9c.

## 3. Fase Operativa3.1 Avvio dei Servizi

Sul sistema Kali Linux è stato avviato il tool **INetSim** (come mostrato negli screenshot del terminale), che ha attivato i servizi di simulazione per le porte 80 (HTTP) e 443 (HTTPS).
Questo ha permesso di risolvere le richieste DNS per epicode.internal direttamente verso il server locale.

### 3.2 Analisi dei Livelli Inferiori (Ethernet e IP)

Dall'analisi dei frame intercettati con Wireshark, è stato possibile identificare correttamente i dettagli dell'intestazione (header):
• **Layer 2 (Ethernet II):** Identificati gli indirizzi MAC fisici delle due macchine.

• **Layer 3 (IPv4):** Verificato il corretto instradamento dei pacchetti dal client (.101) al server (.100).

### 4.1 Analisi traffico HTTP

Durante la richiesta verso *http://epicode.internal; http://epicode.internal*, l'analisi del protocollo applicativo (Layer 7) ha mostrato i dati in **chiaro**. Come visibile negli screenshot di Wireshark:

• È possibile leggere l'intero metodo GET / HTTP/1.1.
• Sono visibili gli Header (Host, User-Agent, Accept-Language).
• **Rischio:** Un malintenzionato potrebbe intercettare e leggere il contenuto della comunicazione senza alcuna difficoltà.
Ripetendo l'esercizio con il server HTTPS, il comportamento della rete cambia radicalmente:
• **TLS Handshake:** Wireshark identifica l'inizio della comunicazione tramite il pacchetto Client Hello (versione **TLS 1.2**).
• **Cifratura:** Dopo lo scambio iniziale delle chiavi, il payload della richiesta non è più leggibile in chiaro. Viene visualizzato solo come "Application Data" cifrato.
• **Differenza riscontrata:** Mentre in HTTP potevamo leggere l'URL e i dati richiesti, in HTTPS il contenuto rimane protetto, garantendo confidenzialità e integrità.

### 4.2 Analisi traffico HTTPS

Ripetendo l'esercizio con il server HTTPS, il comportamento della rete cambia radicalmente:
• **TLS Handshake:** Wireshark identifica l'inizio della comunicazione tramite il pacchetto Client Hello (versione **TLS 1.2**).
• **Cifratura:** Dopo lo scambio iniziale delle chiavi, il payload della richiesta non è più leggibile in chiaro. Viene visualizzato solo come "Application Data" cifrato.
• **Differenza riscontrata:** Mentre in HTTP potevamo leggere l'URL e i dati richiesti, in HTTPS il contenuto rimane protetto, garantendo confidenzialità e integrità.

## 5. Conclusioni

L'esercitazione ha dimostrato con successo l'importanza dei protocolli crittografici.
• La configurazione degli indirizzi IP e dei servizi DNS è risultata corretta per la simulazione.
• L'intercettazione con Wireshark ha confermato che il traffico **HTTP è vulnerabile** all'ascolto (sniffing), poiché trasmette informazioni leggibili.
• Il traffico **HTTPS implementa TLS**, rendendo impossibile per un osservatore esterno leggere il contenuto delle comunicazioni tra client e server.